Using SFS, a Simple File System for the PDP-1

This document describes SFS and how to use it

What is SFS?

It's a minimal file system that uses the Type 23 Parallel Drum for storage of files. It consists of assembler code that is preloaded or included in programs that want to use it.

IOTs 61-63 must be installed to provide the drum access, and the **am1** assembler must be used to handle the extended memory access.

Features

A directory of files is kept on track 31 of the drum that includes the name, the drum location, and the length. A file is limited to 16383 18-bit words. The size of a file is specified by the user code, the SFS code manages the directory and drum usage, and handles transfers to and from the user space and the drum.

User code can get a count of files, get a listing of files, get information about the size of a file, create a new file, delete an existing file, update the file type and user flags, and transfer data to and from a file.

The code that manages the drum and transfers resides in extended memory, bank 15 by default, but this can be changed by reassembling the code.

The code is written in **am1**, the advanced assembler for the PDP-1.

Numbers below that are octal have a leading 0, decimal numbers do not.

The directory structure

A directory entry consists of a 9 character name, a size, a drum track, a drum starting address, and a status word. The structure is:

```
word use
0    name, first 3 characters
1    name, next 3 characters
2    name, last 3 characters
3    file size, 0-16383 decimal
4    drum track and location of the beginning of the file
5    user file type and value word, type in bits 0-5, value in 6-17
```

The name is stored as 9 flexo/concise characters. The full 9 characters are used. If the name is being constructed, pad any trailing characters with flexo 00, space.

In order to avoid using up name space with shift characters, it is advised that the names be in all lower shift.

The status word consists of 12 bits of of a user-specified value in the low part of the word and 6 bits of file type in the high part of the word. The user value can be used for any purpose, such as a version number. The file code does not

attach any meaning to it. The file type has meaning only to the user, the only use the file code makes of it is for filtering a file list by file type.

Space is allocated on the drum for up to 128 directory entries, this can be changed in the code. For efficiency, the directory table is loaded from the drum into the extended memory bank with the management code. It is rewritten to the drum whenever a change is made, such as adding or deleting a file entry.

Additionally, information about drum use is stored following the directory entries.

```
word use
0    next available track/address location
1    count of files, same as number of used directory entries
```

The directory table, etc. is stored at the beginning of the designated track, the remainder of the track is available for storage.

Directory management logic

The block of memory containing the directory data is treated as an array of entries. When an entry is added, it goes into the next entry position following the last and the entry count incremented.

When a file is added, it is allocated following the last file. This, in combination with the directory entry management, means that the directory list will always be in order of ascending drum location.

For allocation purposes, the drum is considered one large, contiguous space of 131,072 18-bit words, track boundaries are transparent.

When a file is deleted, the directory array is packed to remove the hole that would otherwise remain and the entry count is decremented. The drum space is not immediately reclaimed, a pack operation must be done.

The operations

All of the management routines are called by first enabling extended memory then using a farjda() call defined in the memory.ah include file. Extended memory can be disabled after the call returns if desired. All file data is transferred via the high speed channel dma by the drum and will be available to the program when the call returns.

Errors are returned in the IO register, all are negative and so can be easily tested.

First, include <filesystem.ah> in your code as well as <memory.ah>.

The examples below use macros from memory.ah, but of course instead of farjda(), explicit can always be used. However, remember that a jda by itself will **not** work with extended memory. The macro expands into:

```
farjda(foo:2)

dac i [foo:2]
jsp i [foo:2 + 1]
```

- fileCount - get the number of allocated files

  The number of allocated files is returned in the IO register. No errors are possible.

```
farjda(fileCount)
```

- fileList - get a file listing filtered by type

  The IO register contains the address of a memory area to store the list of files. The address must be a full 16 bit address.

  This area must be at least 3 * (max files that could be returned) words long. To be safe, reserve 3 * 128, 364 words, 0534, or 3 * the maximum number of directory entries defined.

  The AC register contains the 6 bit filetype, 0 means *all files*.

  On return, the IO register will have the count of names returned, no errors are possible.

```
lio bufaddress
law type
farjda(fileList)
```

- fileFind - look up a file

  The IO register contains the address of a 3 word memory area containing the name of the file. The address must be a full 16 bit address.

  On return, the IO register will contain the assigned file number, which is the index of the directory entry in the directory list, or an error code.

```
farjda(fileFind)

Memory block:
name 1st 3 chars
name 2nd 3 chars
name 3rd 3 chars
```

  Example:

```
eem
lio [mem:.
farjda(fileFind)

mem,
flexo "ABC"
flexo "D 1"
flexo "2"
```

  This looks up a file named "ABCD 12".

  Possible errors are FILENOTFOUND.

- fileSize - return the size of a file

  The IO register contains the file number as returned by *fileFind* or *fileAllocate*.

On return, the IO register will be the size of the file in words or an error code.

```
lio 017
farjda(fileSize)
```

Possible errors are FILENOTFOUND, FILENUM

- fileAllocate - create a new file and directory entry

  The IO register contains the address of a 5 word memory area containing the information below. The address must be a full 16 bit address.

  On return, the IO register will contain the assigned file number, which is the index of the directory entry in the directory list, or an error code.

```
farjda(fileAllocate)

Memory block:
name 1st 3 chars
name 2nd 3 chars
name 3rd 3 chars
file user file type and user data, as in a directory entry
size to allocate in words
```

  Example:

```
eem
lio [mem:.
farjda(fileAllocate)

mem,
flexo "ABC"
flexo "D"
0
030001
0200
```

  This creates a file named "ABCD" 0200 words long of type 03 with user data 01. On return, the IO register will contain the file number, which is the index in the directory of the file's entry or an error code
  Possible errors are FILETOOBIG, FILEDUPLICATE, FILEPROTECTED.

- fileDelete - delete a file

  The IO register contains the file number to delete, as returned by *fileFind* or *fileAllocate*.

  On return, the IO register will be 0 or an error code.

```
lio [012
farjda(fileDelete)
```

  Possible errors are FILENUM

- fileRead - read all or part of a file into memory

  The AC register contains the file number to read, as returned by *fileFind* or *fileAllocate./* The IO register contains the address of a 3 word memory area containing the address to read the data into, the number of words to read, and the offset in the file to read from./ The addresses must be a full 16 bit address.

  On return, the IO register will be the actual number of words read or an error code.

```
farjda(fileRead)

Memory block:
memory address to load into
number of words to load
offset in file to begin the read from
```

  Example:

```
eem
lio [mem:.
law filenum
farjda(fileRead)

mem,
mybuf:.
0200
01000
```

  This will read 0200 words beginning at file offset 01000, or the file size if the file is smaller, into the memory at address mybuf.

  If the read size is greater than the file size, only the number of words in the file will be returned. If the read size plus the buffer address exceeds 4095, 07777, only the number of words that would not exceed the end of the memory bank will be returned.

  Possible errors are FILENOTFOUND, FILENUM.

- fileWrite

  The AC register contains the file number to write to, as returned by *fileFind* or *fileAllocate./* The IO register contains the address of a 3 word memory area containing the address to write the data from, the number of words to write, and the offset in the file to write to. The addresses must be a full 16 bit address.

  On return, the IO register will be 0 or an error code.

```
farjda(fileWrite)

Memory block:
memory address to write from
number of words to write
offset in file to write to
```

  Example:

```
eem
lio [mem:.
law filenum
farjda(fileWrite)

mem,
mybuf:.
0200
0
```

This will write 0200 words into the the file at offset 0 from address mybuf.

If the write size plus the offset is greater than the file size, an error will be returned and no data will be written.

Possible errors are FILETOOBIG, FILENUM.

- filePack

  Reclaim deleted space on the drum by moving data past a deleted area down to eliminate the hole, repeating this to eliminate all holes. Directory entries are updated appropriately. Note that this is a relatively expensive operation as it potentially requires moving significant words on the drum.

  No arguments are passed, no errors are possible.

```
farjda(filePack)
```

Errors

The error codes that can be returned are:

```
- FILETOOBIG - attempting to create a file that is too big for the drum location given,
    or write past the end of a file or past the end of a memory bank
- FILEDUPLICATE - attempting to create a file, but a file of the same name already exists
- FILEPROTECTED - attempting to create a file that would overwrite the directory area
- FILENOTFOUND - attempting to look up a file but it does not exist
- FILENUM - the file number does not refer to an existing directory entry
```

Notes

- a file can be read or written anywhere within the file and with any number of bytes, as long as there is no attempt to write past the end of a file
- the data area for a file on the drum is not initialized to any particular value
- extended memory mode will be on after all returns; it can be disabled after the return
- a read will stop the transfer if the end of the target memory bank, address 07777, is reached
- a write with a word count that would go past the end of the source memory bank, address 07777, will fail and no data will be written